Campaign Digital Platform — Architecture Documentation

Project: Campaign Website & Digital Platform **Prepared for:** Mr El **Prepared by:** Olukayode Soliu — Myri Digital Services **Date:** 2026-06-16 **Document status:** Living document — updated as the build progresses.

1. Purpose of this document

This document describes the technical architecture of the Campaign Digital Platform: its components, how they fit together, the data they hold, how requests flow through the system, how it is deployed, and how it is secured. It is the engineering reference for the build team and a technical record for the campaign.

It is **grounded in the approved proposal scope** (donation & mobilization engine, content hub, supporter database, admin dashboard) and reflects the current state of delivery.

2. Current project status

Stage	Scope	Status
Stage 1 — Discovery & foundations	Requirements, UX/UI design, compliance review (NDPA + campaign-finance), payment-gateway account setup	**Complete**
Stage 2 — Frontend application	Next.js PWA shell, page structure, design system, donation/forms UI, content rendering	**In progress — completing tomorrow**
Stage 3 — Donation service	Paystack/Flutterwave, recurring giving, receipts, goal thermometer	Upcoming
Stage 4 — Notification + Supporter/Volunteer + CMS services	Broadcasts (SMS/WhatsApp/email), supporter DB, events/RSVP	Upcoming
Stage 5 — Voter toolkit, multilingual scaffolding, admin dashboard	Toolkit content, i18n, admin panels	Upcoming
Stage 6 — Hardening, testing, deployment, training, launch	Security, QA, CI/CD to production, team training	Upcoming

> **As of today:** Stage 1 is signed off. The frontend application (Stage 2) is built and finalising for handover tomorrow; backend services are scaffolded against the contracts in §5 and wired in over

> the following stages.

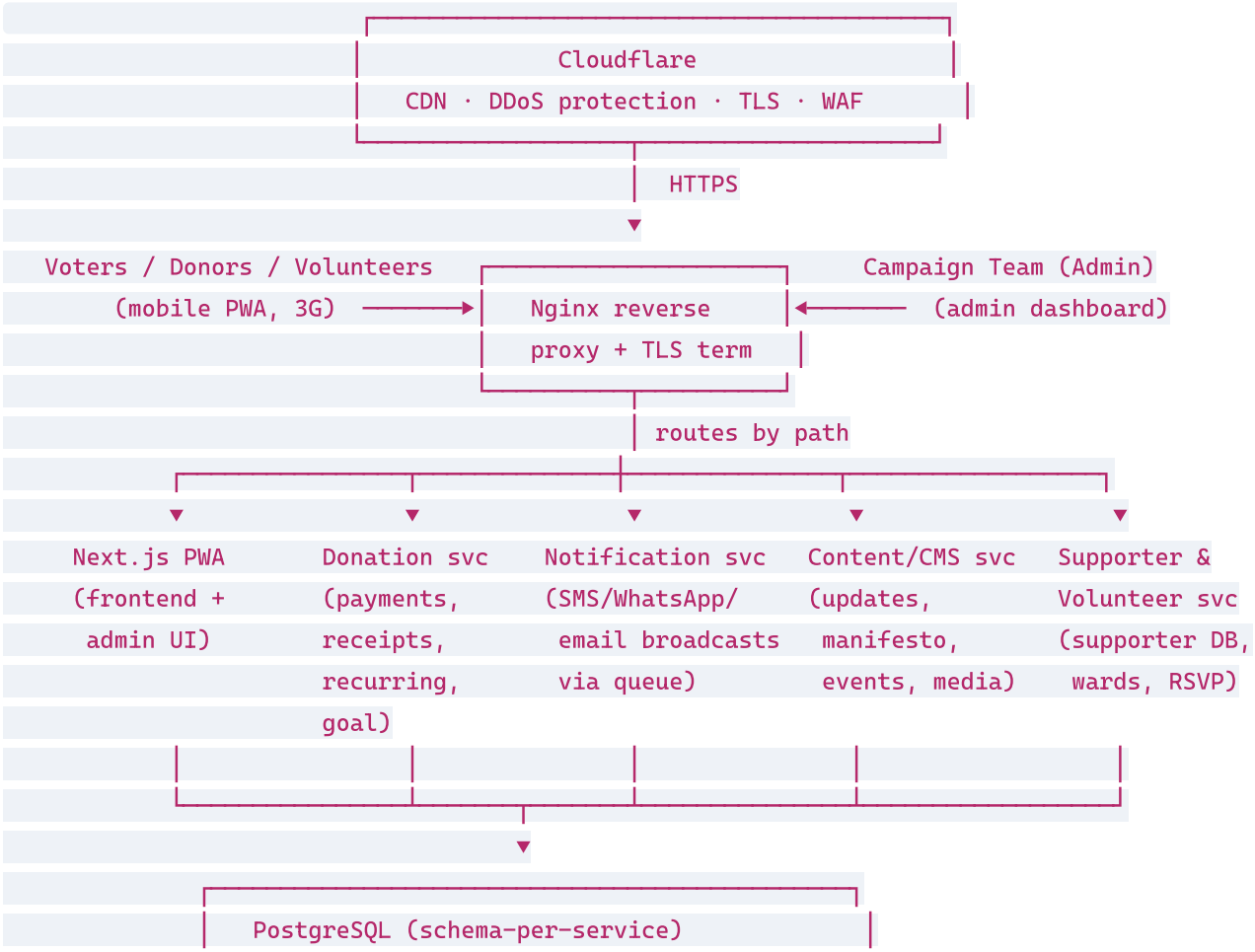
---

3. Architecture principles

1. **Mobile-first, low-data.** The Nigerian voter is on a low-end Android on 3G. Everything is measured against fast first paint, small payloads, and an installable PWA.
2. **Isolated services.** The platform is split into independent backend services so a traffic surge or a failure in one (e.g. a rally driving load) cannot take down the donation flow.
3. **The donation flow is sacred.** Payments are the highest-value, lowest-tolerance path. It is designed to stay up and stay correct even when everything else is under load.
4. **Server is the source of truth for money and consent.** Amounts, payment status, and opt-in/opt-out are never trusted from the client.
5. **Campaign team self-sufficiency.** Content, supporters, and broadcasts are managed through the admin dashboard — no developer needed for day-to-day operation.
6. **Compliant by design.** Consent capture, donor records, and the "Paid for by" disclaimer are built into the flows, not bolted on.

---

4. System context (high level)



```
              │   Redis + BullMQ (queues & cache)                │
              └──────────────────────────────────────────────────┘

  External integrations:  Paystack / Flutterwave · Bulk SMS gateway ·
                          WhatsApp Business API · Transactional email provider
```

---

5. Components

5.1 Frontend — Next.js PWA *(Stage 2, completing tomorrow)*

- **Stack:** Next.js (React) + Tailwind CSS, configured as an installable Progressive Web App.
- **Responsibilities:**
- Public site: candidate story / vision / manifesto, updates/news hub, events calendar, voter toolkit, donation page with live "₦ raised of goal" thermometer.
- Capture forms (email / SMS / WhatsApp opt-in, volunteer sign-up, event RSVP).
- Admin dashboard UI (content editing, supporter lists, broadcast composer, analytics).
- **Key characteristics:**
- Server-side rendering / static generation for fast first paint and SEO on public pages.
- PWA service worker for installability and resilient repeat visits on poor networks.
- Multilingual-ready structure (English + Pidgin / relevant local language).
- Talks to backend services only through the Nginx-routed API; no secrets in the client.

5.2 Donation service *(Stage 3)*

- Initialises and verifies payments via **Paystack / Flutterwave** (cards, **bank transfer, USSD**, Apple/Google Pay).
- Suggested amounts, **recurring (monthly) giving**, automatic receipts.
- Maintains the **fundraising goal + running total** that powers the public thermometer.
- Optional **donor wall** (respecting opt-out).
- **Money rules:** amount and currency are server-computed; payment status is confirmed by **gateway webhook re-verification**, never trusted from the browser. Webhooks are idempotent.

5.3 Notification service *(Stage 4)*

- Sends **SMS, WhatsApp, and email** broadcasts and transactional messages (receipts, event reminders).
- Backed by **Redis + BullMQ** queues for reliable, retryable, rate-limited delivery — a surge of messages never blocks web requests.
- Consumes events from other services (e.g. "donation succeeded" → receipt; "RSVP created" → reminder schedule).

5.4 Content / CMS service *(Stage 4)*

- Stores and serves **updates/press releases, manifesto/policy positions, events, and media**.
- Edited by the campaign team through the admin dashboard — no developer required.
- Serves content to the frontend for SSR/ISR rendering.

5.5 Supporter & Volunteer service *(Stage 4)*

- The **central supporter database**: contacts, consent state, channels (email/SMS/WhatsApp).
- Volunteer registration and **organisation by ward / polling unit**.
- **Events RSVP** records, feeding the notification service for reminders.
- Source of audience segments for broadcasts.

5.6 Admin dashboard

- Delivered as authenticated routes within the Next.js app, backed by the services above.
- Auth: email/password login with hashed credentials and server-side sessions/JWT; admin-only routes gated on the server. Roles can be extended (e.g. editor vs. admin) as the team grows.
- Surfaces analytics: donations, traffic, and **which channels convert**.

---

6. Data model (proposed, PostgreSQL)

One PostgreSQL instance with a **schema (logical DB) per service** — cost-efficient on a single VPS while preserving service ownership boundaries. Indicative core tables:

Donation service - `donations` (id, donor_ref, amount, currency, channel, status, gateway, gateway_ref, is_recurring, created_at) - `recurring_subscriptions` (id, donor_ref, amount, interval, status, next_charge_at) - `fundraising_goals` (id, label, target_amount, raised_amount, active) - `receipts` (id, donation_id, number, issued_at)

Supporter & Volunteer service - `supporters` (id, name, email, phone, whatsapp, created_at) - `consents` (id, supporter_id, channel, opted_in, source, captured_at) ← NDPA record - `volunteers` (id, supporter_id, ward, polling_unit, availability, status) - `event_rsvps` (id, event_id, supporter_id, status, created_at)

Content / CMS service - `posts` (id, type[update|press|response], title, slug, body, status, published_at) - `manifesto_sections` (id, title, body, order) - `events` (id, title, location, ward, starts_at, rsvp_enabled) - `media` (id, kind, url, caption)

Notification service - `messages` (id, channel, recipient, template, payload, status, attempts, sent_at) - `broadcasts` (id, channel, segment_query, body, scheduled_at, status, stats)

> Personal data (`supporters` , `consents`) is treated as regulated PII under the NDPA — see §10.

---

7. Key data flows

7.1 Donation

```
Donor (PWA) → Donation svc: create intent (server sets amount/currency)
Donation svc → Paystack/Flutterwave: initialise → return checkout
Donor pays (card / transfer / USSD)
Gateway → Donation svc (webhook): payment event
Donation svc: RE-VERIFY with gateway → mark donation paid (idempotent)
```

```
            → update fundraising goal total (thermometer)
            → emit "donation.succeeded"
Notification svc: consume event → send receipt (email/SMS/WhatsApp)
```

7.2 Supporter capture & broadcast

```
Visitor submits opt-in form → Supporter svc: store supporter + consent (channel, source, time)
Admin composes broadcast → selects segment (e.g. ward = X, opted_in = true)
Notification svc: enqueue jobs in BullMQ → workers send, retry, rate-limit
            → record per-message status; opt-outs suppress future sends
```

7.3 Event RSVP & reminders

```
Supporter RSVPs to event → Supporter svc: store RSVP
Notification svc: schedule reminder jobs (e.g. 24h / 2h before) on the queue
At due time: worker sends SMS/WhatsApp reminder to opted-in attendees
```

---

8. Infrastructure & deployment

Layer	Technology	Role
Edge	**Cloudflare**	CDN, DDoS protection, WAF, TLS at the edge
Compute	**Ubuntu VPS** (4 vCPU / 8 GB)	Hosts all containers
Reverse proxy	**Nginx**	TLS termination, path-based routing to services, static assets
Runtime	**Docker** (Docker Compose)	Each service runs as an isolated container
TLS	**Let's Encrypt**	Auto-renewing certificates
Data	**PostgreSQL**, **Redis**	Persistent store; queues + cache
CI/CD	**GitHub Actions**	Build, test, and deploy on push to main

Topology. Cloudflare → Nginx → containers (Next.js, four services) on one VPS, sharing the PostgreSQL and Redis containers. Services are independently restartable; a crash in one is isolated by Docker and surfaced by monitoring.

Deployment flow (CI/CD). Push to `main` → GitHub Actions runs build + tests → deploys to the VPS → containers rebuilt/restarted → health checks confirm services are up. Repeatable and rollback-friendly.

Environments. `development` (local), and `production` (VPS). Configuration and all secrets (gateway keys, messaging credentials, DB URLs) are injected via environment variables / `.env` on the server — never committed to source control.

---

9. External integrations

Integration	Purpose	Notes
Paystack / Flutterwave	Donations — cards, bank transfer, USSD, Apple/Google Pay	NGN-native; webhook signatures verified; status re-fetched server-side
Bulk SMS gateway	SMS broadcasts & reminders	Sent via Notification service queue
WhatsApp Business API	WhatsApp broadcasts & reminders	Template/opt-in rules respected
Transactional email	Receipts, confirmations, email broadcasts	Separate sending domain

All integrations are reached **only from backend services**; credentials live server-side.

---

10. Security & compliance

Security - TLS everywhere (Let's Encrypt + Cloudflare); HTTPS-only. - Cloudflare CDN + DDoS protection and WAF to survive rally-driven traffic spikes and attacks. - Anti-fraud on donations: server-set amounts, signed/verified webhooks, idempotent payment confirmation, rate limiting on forms and payment endpoints. - Honeypot + timing checks on public forms to deter bots. - Secrets in environment variables only; least-privilege DB users per service. - **Automated backups** of PostgreSQL and **uptime monitoring** with alerting.

Compliance - **Campaign-finance:** donation limits, donor record-keeping/KYC fields, and a clear **"Paid for by the campaign"** disclaimer on the donation flow. - **Data protection (NDPA):** supporter phone numbers and emails are personal data. The platform captures explicit consent (channel + source + timestamp), provides a privacy policy, and honours opt-out across all channels.

---

11. Scalability & reliability

- **Service isolation** means the donation path is unaffected by load on content or messaging.
- **Queue-backed messaging** (BullMQ) absorbs broadcast surges and retries failures without blocking web traffic.
- **Cloudflare caching** offloads static and public content from the origin during spikes.
- The VPS is sized for the MVP; services can be scaled (more workers / a larger VPS, or split onto additional hosts) without re-architecting, because they are already containerised and independent.

---

12. Phase 2 (future, not in current build)

WhatsApp/Telegram chatbot, rally livestreaming, merchandise store, supporter "pledge-to-vote" referral engine, and paid-ad landing pages. The microservice structure means each can be added as a new service without disturbing the donation and mobilization core.

All figures and scope per the approved Campaign Website & Digital Platform proposal (2026-06-12). This architecture document is maintained alongside the build.